

or of Engineering, Engineering DRI |
Grzegorz Bizon	Distinguished Engineer - Technical Lead
Jessie Young	Principal Engineer
David Fernandez	Staff Engineer
Imre Farkas	Staff Engineer
João Pereira	Staff Engineer
Joe Burnett	Staff Engineer
Tomasz Maczukin	Senior Engineer

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/graphql_api/_index.md

{{< engineering/design-document-header >}}

GraphQL is a data query and manipulation language for APIs, and a runtime for fulfilling queries with existing data.

At GitLab we want to adopt GraphQL to make it easier for the wider community to interact with GitLab in a reliable way, but also to advance our own product by modeling communication between backend and frontend components using GraphQL.

We've recently increased the pace of the adoption by defining quarterly OKRs related to GraphQL migration. This resulted in us spending more time on the GraphQL development and helped to surface the need of improving tooling we use to extend the new API.

This document describes the work that is needed to build a stable foundation that will support our development efforts and a large-scale usage of the GraphQL API.

Summary

The GraphQL initiative at GitLab started around three years ago. Most of the work around the GraphQL ecosystem has been done by volunteers that are GraphQL experts.

The retrospective on our progress surfaced a few opportunities to streamline our GraphQL development efforts and to reduce the risk of performance degradations and possible outages that may be related to the gaps in the essential mechanisms needed to make the GraphQL API observable and operable at scale.

Amongst small improvements to the GraphQL engine itself we want to build a comprehensive monitoring dashboard, that will enable team members to make sense of what is happening inside our GraphQL API. We want to make it possible to define SLOs, triage breached SLIs and to be able to zoom into relevant details using Grafana and Elastic. We want to see historical data and predict future usage.

It is an opportunity to learn from our experience in evolving the REST API, for the scale, and to apply this knowledge onto the GraphQL development efforts. We can do that by building query-to-feature correlation mechanisms, adding scalable state synchronization support and aligning GraphQL with other architectural initiatives being executed in parallel, like the support for direct uploads.

GraphQL should be secure by default. We can avoid common security mistakes by building mechanisms that will help us to enforce OWASP GraphQL recommendations that are relevant to us.

Understanding what are the needs of the wider community will also allow us to plan deprecation policies better and to design parity between GraphQL and REST API that suits their needs.

Challenges

Make sense of what is happening in GraphQL

Being able to see how GraphQL performs in a production environment is a prerequisite for improving performance and reliability of that service.

We do not yet have tools that would make it possible for us to answer a question of how GraphQL performs and what the bottlenecks we should optimize are. This, combined with a pace of GraphQL adoption and the scale in which we expect it operate, imposes a risk of an increased rate of production incidents what will be difficult to resolve.

We want to build a comprehensive Grafana dashboard that will focus on delivering insights of how GraphQL endpoint performs, while still empowering team members with capability of zooming in into details. We want to improve logging to make it possible to better correlate GraphQL queries with feature using Elastic and to index them in a way that performance problems can be detected early.

- Build a comprehensive Grafana dashboard for GraphQL
- Build a GraphQL query-to-feature correlation mechanisms
- Improve logging GraphQL queries in Elastic
- Redesign error handling on frontend to surface warnings

Manage volatile GraphQL data structures

Our GraphQL API will evolve with time. GraphQL has been designed to make such evolution easier. GraphQL APIs are easier to extend because of how composable GraphQL is. On the other hand this is also a reason why versioning of GraphQL APIs is considered unnecessary. Instead of versioning the API we want to mark some fields as deprecated, but we need to have a way to understand what is the usage of deprecated fields, types and a way to visualize it in a way that is easy to understand. We might want to detect usage of deprecated fields and

notify users that we plan to remove them.

- Define a data-informed deprecation policy that will serve our users better
- Build a dashboard showing usage frequency of deprecated GraphQL fields
- Build mechanisms required to send deprecated fields usage in Service Ping

Ensure consistency with the rest of the codebase

GraphQL is not the only thing we work on, but it cuts across the entire application. It is being used to expose data collected and processed in almost every part of our product. It makes it tightly coupled with our monolithic codebase.

We need to ensure that how we use GraphQL is consistent with other mechanisms we've designed to improve performance and reliability of GitLab.

We have extensive experience with evolving our REST API. We want to apply this knowledge onto GraphQL and make it performant and secure by default.

- Design direct uploads for GraphQL
- Build GraphQL query depth and complexity histograms
- Visualize the amount of GraphQL queries reaching limits
- Add support for GraphQL ETags for existing features

Design GraphQL interoperability with REST API

We do not plan to deprecate our REST API. It is a simple way to interact with GitLab, and GraphQL might never become a full replacement of a traditional REST API. The two APIs will need to coexist together. We will need to remove duplication between them to make their codebases maintainable. This symbiosis, however, is not only a technical challenge we need to resolve on the backend. Users might want to use the two APIs interchangeably or even at the same time. Making it interoperable by exposing a common scheme for resource identifiers is a prerequisite for interoperability.

- Make GraphQL and REST API interoperable
- Design common resource identifiers for both APIs

Design scalable state synchronization mechanisms

One of the most important goals related to GraphQL adoption at GitLab is using it to model interactions between GitLab backend and frontend components. This is an ongoing process that has already surfaced the need of building better state synchronization mechanisms and hooking into existing ones.

- Design a scalable state synchronization mechanism
- Evaluate state synchronization through pub/sub and websockets
- Build a generic support for GraphQL feature correlation and feature ETags
- Redesign frontend code responsible for managing shared global state

Iterations

In the scope of the blueprint

1. GraphQL API architecture
 1. Build comprehensive Grafana dashboard for GraphQL
 1. Improve logging of GraphQL requests in Elastic
 1. Build GraphQL query correlation mechanisms
 1. Design a better data-informed deprecation policy

Future iterations

1. Build a scalable state synchronization for GraphQL
1. Add support for direct uploads for GraphQL
1. Review GraphQL design choices related to security

SECTION: engineering/architecture/design-documents/group_and_project_deletions/_index.md

Summary

This architecture blueprint outlines a comprehensive redesign of GitLab's deletion flow for groups and projects.

The current system presents significant challenges, including accidental permanent deletions, high support burden for recovery requests, and locked namespace paths.

Our proposal introduces a consistent, user-friendly deletion experience across all pricing tiers with a "pending deletion" state, recovery options, and automatic namespace renaming. We recommend implementing this vision in three iterations, enhancing protection against data loss while maintaining user control.

These changes will significantly reduce support requests, mitigate accidental deletions, and improve the overall experience for all GitLab users regardless of pricing tier.

Motivation

The current GitLab deletion flow has several critical shortcomings that negatively impact both users and internal teams.

Projects in personal namespaces lack deletion protection entirely, while protection for group-based projects is inconsistent across pricing tiers.

When users accidentally delete content, they have limited recourse, leading to unnecessary support requests and infrastructure team involvement for manual recoveries.

Additionally, some namespace paths remain locked while pending deletion, preventing users from reusing paths immediately.

Goals

1. Create a consistent deletion experience across all pricing tiers and namespace types (group, project, personal namespace)
2. Reduce accidental deletions by implementing a "pending deletion" state for all entities

3. Decrease support and infrastructure team burden by enabling self-service recovery options
4. Free up namespace paths immediately upon deletion through automated renaming
5. Improve user experience by providing clear visibility and control over the deletion process
6. Separate the concepts of "moved to bin" (revocable) and "permanent deletion" (irrevocable) with appropriate UI

Non-Goals

- Modifying how projects are associated with groups or user namespaces

Proposal

We propose implementing a unified deletion flow across all GitLab environments and pricing tiers, with a phased approach consisting of three iterations:

1. Iteration 1: Basic Recovery Framework

- Implement consistent pending deletion period before permanent deletion for all pricing tiers
- Add self-service recovery options for all deleted items
- Standardize UI for deletion status across user and group namespaces
- Implement consistent automated renaming to free up paths immediately

2. Iteration 2: Extended Deletion Protection

- Extend pending deletion period from 7 to 30 days on SaaS
- Implement email notifications for deletion events and upcoming permanent deletions
- Update scheduling system to support longer retention period

3. Iteration 3: Enhanced Deletion Interface

- Introduce dedicated "Bin" section in UI for managing deleted items
- Create clear separation between "Move to Bin" and "Delete Permanently" actions
- Implement bulk actions for trash management (restore multiple, empty trash)
- Add filtering and sorting capabilities in the "Bin" UI

Find a visual reference of this proposal in Figma.

Design and Implementation Details

Current State

GitLab's deletion flow currently operates differently based on:

- Pricing tier (free vs. premium/ultimate)
- Entity type (project vs. group)
- Namespace type (user vs. group)

On premium and ultimate tiers, projects and groups enter a "pending deletion" state for 7 days, during which users can either recover or permanently delete them.

On the free tier, deleted items are immediately hidden from users and only accessible to admins.

Projects in user namespaces are deleted permanently without any recovery period.

New Unified Deletion Flow

Core Components

1. Deletion Initiation

- When a user initiates deletion, the item and all its child items, if any, move to "pending deletion" state
- System automatically renames the namespace to free up the original path
- Item becomes inaccessible to regular operations but visible in a new "Bin" UI
- All child items are treated like the parent item that is deleted

2. Pending Deletion State

- Items remain in this state for a configurable period (7 days initially, 30 days in Iteration 2)
- Clear UI indicators show deletion status and remaining time until permanent deletion
- Email notifications sent at key milestones (initial deletion, 7 days before permanent deletion)
- Items pending deletion still count towards usage quotas

3. Recovery Mechanism

- Self-service recovery option available to users who have appropriate permissions
- Recovery action restores item to active state with original content, including its child items
- System handles potential namespace conflicts during recovery

4. Permanent Deletion

- In iteration 1: Available to users with appropriate permissions
- In iteration 3: Available through separate "Delete Permanently" action in Bin

5. Bin Interface

- Dedicated section showing all deleted items the user has access to
- Features for filtering, sorting, and bulk actions
- Clear distinction between temporary deletion and permanent deletion

Technical Implementation Considerations

1. Database Changes

- Add fields to track deletion state, scheduled permanent deletion date, and original path
- Implement soft deletion pattern across all relevant entities

2. API Enhancements

- Extend API to support new deletion states and operations
- Implement endpoints for recovery, scheduled deletion, and bin management

3. Namespace Handling

- Implement automatic suffix appending to free up namespace paths

- Create conflict resolution strategy for recovery scenarios
- Ensure proper validation for all namespace operations

4. Permission Model

- Define permission requirements for viewing, recovering, and permanently deleting items
- Implement permission checks consistently across UI and API
- Support role-based access control for deletion operations

5. Notification System

- Create templates for deletion-related notifications
- Implement scheduling for time-based notification triggers
- Provide clear action paths from notification emails

User Experience Flow

User Moving a Project/Group to Bin

1. User selects "Move to Bin" option for project/group
2. System displays confirmation dialog explaining the process
3. Upon confirmation, system:
 - Moves item to pending deletion state
 - Renames namespace by appending unique identifier
 - Displays success message with link to Bin
 - Sends confirmation email with recovery information

User Recovering a Project/Group

1. User accesses Bin
2. Locates desired item using search/filter
3. Selects "Recover" option
4. System handles recovery and potential namespace conflicts
5. Displays success message with link to recovered item

Permanent Deletion

1. Occurs automatically after configured period (7/30 days)
2. Admin can trigger early by using admin interface (all iterations)
3. User can trigger early by using "Delete Permanently" in Inactive tab (iteration 1)
4. User can trigger early by using "Delete Permanently" in Bin (iteration 3)
5. System requires confirmation for permanent deletion
6. Notification sent after successful permanent deletion

Alternative Solutions

Permanent Deletion Restriction

Only admin users would retain the ability to permanently delete content before the scheduled deletion date. This approach significantly changes how users interact with the deletion process and creates several important trade-offs.

Pros

- Enhanced data protection: Dramatically reduces instances of accidental permanent data loss by removing the capability for users to perform immediate permanent deletions
- Reduced support burden: Fewer accidental deletion tickets to handle, freeing up support resources for other issues
- Simplified user interface: Users encounter only one deletion action rather than having to choose between soft and permanent deletion
- Consistent recovery experience: All users gain the same extended 30-day recovery window regardless of tier or namespace type

Cons

- Increased storage costs: Extending the pending deletion period from 7 to 30 days increases storage requirements approximately fourfold
- Loss of user autonomy: Users lose direct control over their content's lifecycle, potentially causing frustration
- Admin workload increase: Admins may become bottlenecks if many users request early permanent deletions
- Potential privacy concerns: Users unable to immediately and permanently remove sensitive data without admin involvement
- Implementation complexity: Requires additional permission controls and admin interface to manage deletion requests

SECTION: engineering/architecture/design-documents/group_and_project_operations_and_state_management/_index.md

{{< engineering/design-document-header >}}

Summary

This blueprint proposes a unified state management and tracking system for GitLab namespaces (groups and projects), as well as guidelines on making state-related operations asynchronous. Currently, groups and projects implement state management (deletion, archival, transfer) as separate features with inconsistent data representations and no historical tracking. The proposed solution introduces a centralized state management system using `namespaces.state` and `namespacedetails.statemetadata` to provide consistent state tracking, metadata storage, and historical records across all namespace types.

Motivation

Groups and Projects currently have inconsistent state management implementations that create several problems:

Current Issues:

- No consistency in group state management
- No consistency in project state management
- No consistency between group and project state management
- State in descendants is sometimes inferred from ancestors inconsistently
- No state history tracking. For instance, it's impossible to know when a project was archived, then unarchived, or when a group was transferred from another namespace
- Different data models for similar operations (for example `groupdeletionschedules` vs `projects.markedfordeletionat` to track the "scheduled for deletion" state)
- Performance issues with long-running synchronous operations (99.95th percentile: group transfer 51s, project transfer 27s)

Business Impact:

- Poor user experience due to inconsistent behavior and bugs
- Poor user experience due to performance bottlenecks causing timeouts
- Increased load on the Support and Engineering teams to resolve operations that failed due to timeouts or bugs
- Difficulty in auditing and compliance
- Maintenance overhead from duplicated and inconsistent code

Goals

- Establish a unified state management system for all namespace types
- Provide consistent APIs and behavior across groups and projects
- Enable state history tracking and audit capabilities
- Improve performance by making appropriate operations asynchronous
- Support metadata tracking (who initiated actions, error states, inheritance)
- Reduce code duplication and maintenance overhead
- Enable better observability and debugging capabilities

Non-Goals

- Complete rewrite of existing functionality (iterative migration approach)
- Changes to user-facing APIs or UI in the initial implementation
- Migration of non-state related group/project consolidation work
- Performance optimizations unrelated to state management

Proposal

Introduce a centralized namespace state management system and asynchronous operation guidelines.

Core State Model

New Attributes:

- `namespaces.state` (SMALLINT) - State identifier
- `namespacedetails.statemetadata` (JSONB) - Associated metadata

State Metadata Structure:

```

json
{
  "lastupdatedat": "2025-05-26T10:00:00Z",
  "lastchangedbyuserid": 12345,
  "inheritedfromnamespaceid": 67890,
  "lasterror": "Transfer failed: insufficient permissions"
}

```

State definitions

State Integer	State Name	Description
0	active	Normal operational state
1	archived	Archived but recoverable
2	ancestorarchived	An ancestor of this namespace is archived
3	deletionscheduled	Marked for deletion with a grace period
4	ancestordeletionscheduled	An ancestor of this namespace is marked for deletion with a grace period
5	creationinprogress	Currently being created
6	deletioninprogress	Currently being deleted
7	transferinprogress	Currently being transferred

State transitions

```

mermaid
stateDiagram-v2
    direction LR
    ac: active
    ar: archived
    aar: ancestorarchived
    ds: deletionscheduled
    ads: ancestordeletionscheduled
    cip: creationinprogress
    dip: deletioninprogress
    tip: transferinprogress
    [] --> ac
    [] --> cip
    note right of cip
        Temporary state while operation is in progress
    end note
    cip --> ac
    note right of ac
        Permanent state until human action
    end note
    ac --> ar: archive
    note right of ar
        Permanent state until human action
    end note
    ac --> aar: archive

```

```

note right of aar
  Permanent state until ancestor is unarchived
end note
aar --> ac: unarchive
ar --> ac: unarchive
ar --> ds: scheduleddeletion
ds --> ac: restore
ac --> ads: ancestorscheduleddeletion
aar --> ads: ancestorscheduleddeletion
note right of ads
  Temporary state until operation on ancestor starts
end note
ads --> ac: restore
ads --> dip: deleteimmediately
note right of ds
  Temporary state until operation starts
end note
ac --> ds: scheduleddeletion
note right of dip
  Temporary state while operation is in progress
end note
ds --> dip: deleteimmediately
note right of tip
  Temporary state while operation is in progress
end note
ac --> tip: transferstart
tip --> ac: transferdone
ac --> []
dip --> []

```

Asynchronous operation guidelines

Operations must be asynchronous if they meet any of the following criteria:

- P99.95 performance exceeds 10 seconds
- Operation involves cascading changes to descendants
- Operation requires external service calls or integrations
- Operation involves bulk database writes, deletes, or migrations
- Risk of creating database locks that affect concurrent operations
- Memory-intensive operations that could impact system resources

Implementation requirements

All asynchronous operations must provide:

- State transition to appropriate inprogress state
- Immediate acknowledgment of the successful request and ongoing operation
- Progress indicators where technically feasible
- Completion notifications through appropriate channels (activity, notification center, email)

- Comprehensive error handling with user-facing error messages
- Rollback capabilities for failed operations

Current operations requiring async implementation

Based on current performance metrics:

- Group transfers (P99.95: 51s) - Priority 1
- Project transfers (P99.95: 27s) - Priority 1
- Upcoming group archiving

Design and implementation details

Database layer

Phase 1: Add new columns and indices

ruby

```
class AddStateToNamespaces < Gitlab::Database::Migration[2.3]
  milestone '18.1'
  disableddltransaction!
```

```
  def change
    addcolumn :namespaces, :state, :smallint
  end
end
```

```
class AddStateMetadataToNamespaceDetails < Gitlab::Database::Migration[2.3]
  milestone '18.1'
  disableddltransaction!
```

```
  CONSTRAINTNAME = 'checknamespacedetailsstatemetadashaishash'
```

```
  def up
    withlockretries do
      addcolumn :namespacedetails, :statemetadata, :jsonb, default: {}, null: false
    end

    addcheckconstraint :namespacedetails, "(jsonb_typeof(statemetadata) = 'object')",
    CONSTRAINTNAME
  end
```

```
  def down
    removecheckconstraint :namespacedetails, CONSTRAINTNAME

    withlockretries do
      removecolumn :namespacedetails, :statemetadata
    end
  end
end
```

```

class AddIndexToStateOnNamespaces < Gitlab::Database::Migration[2.3]
  milestone '18.1'
  disableddltransaction!

  INDEXNAME = 'indexnamespacesonstate'

  def up
    addconcurrentindex :namespaces, :state, name: INDEXNAME
  end

  def down
    removeconcurrentindexbyname :namespaces, INDEXNAME
  end
end

```

Phase 2: Create namespace state updates table

```

ruby
class CreateNamespaceStateUpdates < Gitlab::Database::Migration[2.3]
  disableddltransaction!
  milestone '18.1'

  CONSTRAINTNAME = 'checknamespacestateupdatesmetadatahash'

  def up
    createtable :namespacestateupdates do |t|
      t.timestampswithtimezone null: false
      t.smallint :fromstate, null: false
      t.smallint :tostate, null: false
      t.references :namespace,
        null: false,
        foreignkey: { ondelete: :cascade },
        index: true
      t.jsonb :metadata, default: {}, null: false
    end

    addcheckconstraint :namespacestateupdates, "(jsonb_typeof(metadata) = 'object')",
    CONSTRAINTNAME
  end

  def down
    droptable :namespacestateupdates
  end
end

```

Model layer

New Concern: Namespaces::Stateful

```

ruby
module Namespaces
  module Stateful
    extend ActiveSupport::Concern
    included do
      has_many :namespacestateupdates, dependent: :deleteall

      STATES = {
        active: 0,
        archived: 1,
        ancestorarchived: 2,
        deletionscheduled: 3,
        ancestordeletionscheduled: 4,
        creationinprogress: 5,
        deletioninprogress: 6,
        transferinprogress: 7
      }.withindifferentaccess.freeze

      statemachine :state, initial: :active, initialize: false do
        event :creationdone do
          transition creationinprogress: :active
        end

        event :archive do
          transition [:active] => :archived
        end

        event :ancestorarchive do
          transition [:active, :archived, :deletionscheduled, :ancestordeletionscheduled] =>
:ancestorarchived
        end

        event :unarchive do
          transition [:archived, :ancestorarchived] => :active
        end

        event :scheduleddeletion do
          transition [:active, :archived] => :deletionscheduled
        end

        event :ancestorscheduleddeletion do
          transition [:active, :archived, :ancestorarchived] => :ancestordeletionscheduled
        end

        event :restore do
          transition [:deletionscheduled, :ancestordeletionscheduled] => :active
        end

        event :deletionstart do
          transition [:deletionscheduled, :ancestordeletionscheduled] => :deletioninprogress
        end
      end
    end
  end
end

```

```

end

event :transferstart do
  transition active: :transferinprogress
end

event :transferdone do
  transition transferinprogress: :active
end

aftertransition any => any do |namespace|
  namespace.runaftercommit do
    namespacestateupdates.create!(fromstate: statewas, tostate: state, metadata:
statemetadata)
  end
end

state :active, value: STATES[:active]
state :archived, value: STATES[:archived]
state :ancestorarchived, value: STATES[:ancestorarchived]
state :deletionscheduled, value: STATES[:deletionscheduled]
state :ancestordeletionscheduled, value: STATES[:ancestordeletionscheduled]
state :creationinprogress, value: STATES[:creationinprogress]
state :deletioninprogress, value: STATES[:deletioninprogress]
state :transferinprogress, value: STATES[:transferinprogress]
end
end

classmethods do
  def noninheritablestate?(state)
    [:creationinprogress, :deletioninprogress, :transferinprogress].include?(state)
  end
end

def changestate!(newstate, changedbyuser:, inheritedfromnamespace: nil)
  state = newstate

  namespacedetails.statemetadata = {
    lastupdatedat: Time.current
    lastchangedbyuserid: changedbyuser.id
    inheritedfromnamespaceid: inheritedfromnamespace&.id
  }.compact

  save!
end

def statemetadata
  namespacedetails&.statemetadata || {}
end
end

```

end

New model: Namespaces::StateUpdate

Introduce namespacestateupdates table to track all state transitions:

```
ruby
module Namespaces
  class StateUpdate < ApplicationRecord
    self.table_name = 'namespacestateupdates'

    belongs_to :namespace
  end
end
```

State management service:

```
ruby
module Namespaces
  class StateManagementService
    def initialize(namespace, currentuser, inheritedfromnamespace: nil)
      @namespace = namespace
      @currentuser = currentuser
      @inheritedfromnamespace = inheritedfromnamespace
    end

    def execute!(newstate)
      finalstate =
        case newstate
        when :archived
          @inheritedfromnamespace ? :ancestorarchived : :archived
        when :deletionscheduled
          @inheritedfromnamespace ? :ancestordeletionscheduled : :deletionscheduled
        when :deletioninprogress, :transferinprogress
          newstate
        end
      end

      updatestate!(finalstate)
    end

    private

    def updatestate!(newstate)
      @namespace.changestate!(newstate, changedbyuser: @currentuser, inheritedfromnamespace:
      @inheritedfromnamespace)
```



```

    return if Namespace.noninheritablestate?(newstate)
    return unless @inheritedfromnamespace

    updateddescendantsstate(newstate)
  end

  def updateddescendantsstate(state)
    NamespaceDescendantsStateUpdateWorker.perform_async(namespaceid: @namespace.id,
currentuserid: @currentuser.id, state: state)
  end
end

```

State synchronization service:

```

ruby
module Namespaces
  class StateSynchronizationService
    def initialize(namespace, currentuser)
      @namespace = namespace
      @currentuser = currentuser
    end

    def execute!(newstate)
      namespace.descendants.find_each(batchsize: 100) do |descendant|
        Namespaces::StateManagementService
          .new(descendant, currentuser, inheritedfromnamespace: namespace)
          .execute(newstate)
      end
    end
  end
end

```

Worker layer

State synchronization worker:

```

ruby
module Namespaces
  class NamespaceDescendantsStateUpdateWorker
    include ApplicationWorker

    dataconsistency :always

    sidekiqoptions retry: 3
    include ExceptionBacktrace

    INTERVAL = 2.seconds.toi
  end
end

```

featurecategory :groupsandprojects
idempotent!

```
def perform(namespaceid:, currentuserid:, state:)
  namespace = Namespace.findbyid(namespaceid)
  return unless namespace
```

The namespace state has changed since the job was enqueued, the update will be done by another job.

```
  return unless state == namespace.state
```

```
  currentuser = User.findbyid(currentuserid)
```

```
  withcontext(namespace: namespace) do
    Namespaces::StateSynchronizationService.new(namespace, currentuser).execute(state)
  end
end
end
end
```

Migration strategy

Iteration 1: Infrastructure setup

- Add state column to namespaces
- Create namespacestateupdates table
- Add Namespaces::Stateful concern

Iteration 2: Make transfer async

- Make group/project transfer async by taking advantage of the new transferinprogress state
- Show a "Transfer in progress" badge in the UI as an MVC

Iteration 3: State synchronization

- Ensure state updates with legacy state changes
- Implement bidirectional synchronization
- Add validation and consistency checks

Iteration 4: Backfill historical data

- Backfill state from legacy columns/tables, including inherited states, such as ancestor
- Data validation and cleanup

Iteration 5: Feature migration

- Replace legacy state checks with new system
- Update controllers and services
- Maintain backward compatibility

Iteration 6: Legacy cleanup

- Remove legacy state columns/tables
- Update documentation and tests
- Performance optimization

Performance considerations

Asynchronous operations:

- Group transfers (P1 priority - currently 51s at 99.95th percentile)
- Project transfers (P1 priority - currently 27s at 99.95th percentile)
- Large group archival operations

Optimization strategies:

- Batch updates for descendant state changes
- Background job processing for heavy operations
- Caching for frequently accessed state information
- Database indexes on state and related columns

Alternative solutions

The proposed solution balances complexity with benefits, providing immediate improvements while enabling future enhancements, but the following alternative solutions were also considered.

Alternative 1: Separate state tables per type

Pros:

- Clear separation of concerns
- Type-specific optimizations possible

Cons:

- Maintains current inconsistency
- Duplicated logic and maintenance overhead
- No unified querying capabilities

Alternative 2: Event sourcing approach

Pros:

- Complete audit trail
- Time-travel capabilities
- Excellent for compliance

Cons:

- Significant complexity increase
- Performance overhead for simple state queries
- Steep learning curve for team

Alternative 3: Do nothing

Pros:

- No development effort required
- No migration risks

Cons:

- Performance issues persist (P1 transfer operations)
- Inconsistency continues to create bugs
- Technical debt accumulates
- Poor user experience remains

Metrics and success criteria

Performance targets:

- Reduce P99.95 group transfer time from 51s to <10s
- Reduce P99.95 project transfer time from 27s to <5s
- Maintain deletion operation performance (<2s for scheduling)

Quality metrics:

- Zero state inconsistency bugs after full migration
- 100% state history coverage for audit requirements
- Unified test coverage across all namespace types

Developer experience:

- Single API for all state management operations
- Consistent behavior documentation
- Reduced code duplication (target: 50% reduction in state-related code)

Risks and mitigations

Risk: Data migration complexity

- Mitigation: Iterative approach with rollback capabilities, extensive testing

Risk: Performance impact during migration

- Mitigation: Background migrations, feature flags, monitoring

Risk: API breaking changes

- Mitigation: Maintain backward compatibility, versioned APIs

Risk: State consistency issues

- Mitigation: Bidirectional synchronization during transition, validation checks

SECTION: engineering/architecture/design-documents/image_resizing/_index.md

{{< engineering/design-document-header >}}

Currently, we are showing all uploaded images 1:1, which is of course not ideal. To improve performance greatly, add image resizing to the backend. There are two main areas of image resizing to consider; avatars and content images. The MVC for this implementation focuses on Avatars. Avatars requests consist of approximately 70% of total image requests. There is an identified set of sizes we intend to support which makes the scope of this first MVC very narrow. Content image resizing has many more considerations for size and features. It is entirely possible that we have two separate development efforts with the same goal of increasing performance via image resizing.

MVC Avatar Resizing

When implementing a dynamic image resizing solution, images should be resized and optimized on the fly so that if we define new targeted sizes later we can add them dynamically. This would mean a huge improvement in performance as some of the measurements suggest that we can save up to 95% of our current load size. Our initial investigations indicate that we have uploaded approximately 1.65 million avatars totaling approximately 80 GB in size and averaging approximately 48 KB each. Early measurements indicate we can reduce the most common avatar dimensions to between 1-3 KB in size, netting us a greater than 90% size reduction. For the MVC we don't consider application level caching and rely purely on HTTP based caches as implemented in CDNs and browsers, but might revisit this decision later on. To mitigate performance issues with avatar resizing, especially in the case of self managed, an operations feature flag is implemented to disable dynamic image resizing.

mermaid

sequenceDiagram

autonumber

Requester->>Workhorse: Incoming request

Workhorse->>RailsApp: Incoming request

alt All is true: 1.Avatar is requested, 2.Requested Width is allowed, 3.Feature is enabled

Note right of RailsApp: Width Allowlist: <https://gitlab.com/gitlab-org/gitlab/-/blob/master/app/models/concerns/avatarable.rb>L10

RailsApp->>Workhorse: send-scaled-img: request

Note right of RailsApp: Set send-scaled-img: Header

Workhorse->>Workhorse: Image resizing using Go lib

Workhorse->>Requester: Serve the resized image

```
else All other cases
  RailsApp->>Workhorse: Usual request scenario
  Workhorse->>Requester: Usual request scenario
end
```

Content Image Resizing

Content image resizing is a more complex problem to tackle. There are no set size restrictions and there are additional features or requirements to consider.

- Dynamic WebP support - the WebP format typically achieves an average of 30% more compression than JPEG without the loss of image quality. More details are in this [Google Comparative Study](#)
- Extract first GIF image so we can prevent from loading 10 MB pixels
- Check Device Pixel Ratio to deliver nice images on High DPI screens
- Progressive image loading, similar to what is described in this [article](#) about how to build a progressive image loader
- Resizing recommendations (for example, size and clarity)
- Storage

The MVC Avatar resizing implementation is integrated into Workhorse. With the extra requirements for content image resizing, this may require further use of GraphicsMagik (GM) or a similar library and breaking it out of Workhorse.

Iterations

1. · POC on different image resizing solutions
1. · Review solutions with security team
1. · Implement avatar resizing MVC
1. Deploy, measure, monitor
1. Clarify features for content image resizing
1. Weigh options between using current implementation of image resizing vs new solution
1. Implement content image resizing MVC
1. Deploy, measure, monitor

Who

Proposal:

<!-- vale gitlab.Spelling = NO -->

Role	Who
-----	----
Author	Craig Gomes
Architecture Evolution Coach	Kamil Trzeci-ski
Engineering Leader	Tim Zallmann
Domain Expert	Matthias Kaeppler
Domain Expert	Aleksei Lipniagov

<!-- vale gitlab.Spelling = YES -->

DRIs:

<!-- vale gitlab.Spelling = NO -->

Role	Who
-----	-----
Product	Josh Lambert
Leadership	Craig Gomes
Engineering	Matthias Kaeppler

<!-- vale gitlab.Spelling = YES -->

SECTION: engineering/architecture/design-documents/internal_releases/_index.md

Introduction

In 2023 GitLab launched a new fully managed private solution, GitLab Dedicated, our single tenant

SaaS offering completely managed by GitLab Inc. These internal instances run public GitLab versions limiting the ability to deploy security

fixes before a public disclosure. To guarantee the safety and availability

on single tenant SaaS instances, Delivery has an increasing need to prepare private packages to remediate high-severity issues within specific SLAs.

There is no official release process available to remediate these private instances. Given the GitLab open core

and public by default nature, GitLab releases were designed to be public from the beginning of time.

In case of a high-severity issue release managers must manually operate the release tooling available to remediate the single tenant SaaS instances

within the remediation SLAs, impacting GitLab customers and the ability to remediate them on time:

- Elevated coordination between release managers and all the stakeholders involved.

- The current release tooling lacks the flexibility to perform simultaneous releases, for example, if a vulnerability is found near the

- monthly release week preparation, a release candidate must be created earlier on the development cycle, renouncing features and bug fixes for the active monthly release.

- Existing release process are not adept to remediate single tenant SaaS instances putting at risk their availability and integrity.

Goal

Introduce an internal release GitLab process focused on releasing private packages targeted to

private GitLab SaaS managed instances.

Private packages will contain remediations for the high-severity issues before disclosing them to the general audience. Due to the nature of

GitLab business model addressing internal releases brings unique

long-term challenges that will be driving Delivery-Releases work during the upcoming quarters:

1. Understanding the internal release nature. Introducing a private release on top of the GitLab's open nature by default

is a unique challenge that starts by understanding the problem, vision, scope, and challenges.

The outcome of this first step

is this blueprint that documents the internal release roadmap.

2. Remove the blocking nature of releases. Engineering constraints prevent preparing a patch release in the middle of the

monthly release week preparation. Establishing a private release process requires removing this constraint with flexible tooling

to allow simultaneous releases without leaking vulnerabilities before they're disclosed.

3. Introduce a GitLab Dedicated remediation process into the GitLab release strategy. GitLab SaaS single tenant instances are

remediated by manually operating the critical patch release process. This involves considerable toil from release management, security and

engineering stakeholders. Internal releases aim to upgrade the GitLab Dedicated remediation process to be at the same level as GitLab.com

and self-managed remediation processes.

Key terms

Internal releases: New private release strategy to remediate GitLab SaaS single tenant instances within specific SLAs.

GitLab SaaS single tenant instances: SaaS instances managed by GitLab, Inc. At the moment limited to GitLab Dedicated.

High-severity issues: Bugs or security vulnerabilities defined as ~"severity::1" having an impact on availability, functionality or critical for security on GitLab SaaS single tenant instances.

See release terminology for additional definitions.

Goals and non-goals

Vision

Short-term

Align the internal releases effort into the GitLab Delivery strategy. Outline a vision, effort, and an understanding of the

challenges and requirements to embed an internal release process as part of the GitLab releases. We will deliver this by:

Documenting a blueprint detailing the roadmap and iterative steps to achieve the internal release vision.

Epic: <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/1292>

Medium-term

Release Managers will kick off an automated process to create a private release version with the relevant fixes required to remediate GitLab Dedicated without disclosing the vulnerabilities to the public. After the GitLab Dedicated team has sent an acknowledgement that all instances are upgraded, the publish process is then kicked off. We will deliver this by:

Designing an internal release process to remediate high-severity issues on GitLab SaaS single tenant instances.

Epic: <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/1201>

Bridging the patch release process to account for the current GitLab Dedicated remediation strategy. This will

permit the generation of a private package with a different tag from the public patch.

Epic: <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/1373>

Decouple the current auto-deploy strategy. This will transform release tooling into highly flexible tools and

allow to cleanly separate the private and public parts of this process to ensure GitLab Dedicated is running a patched

version before any fixes are communicated/released publicly.

Epic: <https://gitlab.com/groups/gitlab-com/gl-infra/-/epics/1050>.

Long-term

An internal release process to remediate high-severity issues on GitLab SaaS single tenant instances is part of the GitLab release strategy. Release managers will be able to perform an internal release process to remediate high-severity issues in GitLab SaaS single tenant instances. The outcome of this process will allow GitLab to iteratively deliver fixes to GitLab Dedicated based on the remediation SLAs or when there is a pressing need (short critical remediation targets).

The internal release process should be automated end-to-end, only paging engineers when manual intervention is required, and it should remove the blocking nature of GitLab releases: multiple release types should be performed simultaneously.

Use advanced package deployment strategies to prepare a private package without blocking an existing release process.

Emitting private packages without disclosing the vulnerabilities to the general audience.

Enabling the auto-upgrade of Dedicated tenants via Switchboard API.

Scope

Internal releases are limited to introducing a new release process to SaaS instances managed by GitLab, Inc. At the moment,

this is limited to GitLab Dedicated.

Out of scope

Internal releases are not intended to create releases with custom functionalities that diverge from the standard GitLab product offering.

Internal releases are limited to mitigate critical vulnerabilities or bug fixes that impact the availability of

GitLab SaaS single tenant instances. Issues with lower severity stick to the patch release cadence.

GitLab.com and GitLab self-managed instances are not considered for internal releases. On these instances,

auto-deploy, patch and monthly releases remediate bug and vulnerabilities within the remediation SLAs.

GitLab Cells is not considered for internal releases, subject to change in the future.

Internal release channels for self-managed customers are not considered on the initial roadmap. Product analysis and validation will be performed on upcoming stages.

Proposed plan of action

Aside from understanding the internal releases concept and introducing a new internal release strategy, two

main requirements must be addressed:

1. Remove the blocking nature of GitLab releases.
2. Elevate the GitLab Dedicated remediation process to a first-class citizen.

These two requirements can be addressed simultaneously, the work for each will span throughout multiple quarters.

Remove the blocking nature of GitLab releases

The mitigation of a high-severity issue on GitLab.com requires a fix to be merged into the GitLab security repository

and deployed to the GitLab production environment. This operation breaks the mirroring between the GitLab canonical and security

repositories impeding the monthly release preparation: Performing a monthly release with an undisclosed security vulnerability

tampers our ability to remediate GitLab Dedicated and GitLab self-managed instances. Due to this limitation, patch and monthly releases can't be performed simultaneously without leaking the vulnerability to the public.

The auto-deploy strategy must be adapted to be highly flexible allowing the delivery of packages to all GitLab instances without

breaking the mirroring between GitLab repositories and without initiating a patch release. This work will be done in iterations

spanning multiple quarters:

1. First iteration: Decouple auto-deploy tagging from rollout - Removes the strong coupling between building packages and rolling them out to GitLab.com production environments by establishing two independent processes: one for building packages and another for rolling them out. This iteration paves the way for advanced package selection strategies.
2. Second iteration: Filter and selection of auto-deploy packages - With the decoupling of the auto-deploy process, packages can be configured to be built with specific requirements, for example requiring a minimum SHA in all upcoming packages.
This ability can be used to guarantee the mitigation of production incidents.
3. Third iteration: Ability to create custom auto-deploy packages - The flexibility of auto-deploy should be focused on allowing the creation of packages with experimental features (Ruby/Rails upgrades) or with unmerged changes (security fixes) preventing the mirroring divergence between GitLab repositories.

Introduce a GitLab Dedicated remediation process into the GitLab release strategy

Establish a new private release model for GitLab Dedicated and elevate it to the same standard as GitLab.com and self-managed remediation processes. Considering GitLab open-core nature this is an uncharted territory that will require multiple iterations:

1. First iteration: Bridge the patch release process to account for GitLab Dedicated remediation strategy - Adjusts the release tooling to generate self-managed packages without interfering with the patch release process.
2. Second iteration: Introduce an internal release process - Design and introduce an internal release process to remediate GitLab Dedicated instances.
3. Third iteration: Automate the internal release process end-to-end - With an established internal release process and with the ability to create custom internal packages:
The highly flexible auto-deploy process will be used to deploy security fixes without breaking the mirroring between GitLab repositories.
The internal release process will guarantee predictability and results, and minimize the human factor from the process. The remediation of high severity issues in GitLab Dedicated instances will be automated from start to end by using the Switchboard functionality to perform an auto-upgrade of Dedicated tenants.

SECTION: engineering/architecture/design-documents/kev/_index.md

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

KEV (Known Exploited Vulnerabilities) is a catalog maintained by CISA that identifies vulnerabilities that have been actively exploited in the wild. KEV support in GitLab aims to enhance vulnerability prioritization and remediation efforts by highlighting these high-risk vulnerabilities. The KEV support requirements are outlined in the KEV epic. This document focuses on the technical implementation of KEV support.

KEV data will be sourced from the CISA KEV catalog, which is available as a JSON file on the CISA website. This file is regularly updated by CISA and can be downloaded and processed to extract the latest KEV information. The goal is to make KEV information accessible through the GitLab GraphQL API, visible on vulnerability report and details pages, and usable for filtering and policy setting. The implementation will leverage the existing Package Metadata Database (PMDB, also known as license-db) infrastructure for advisory pull-and-enrichment. The flow is as follows:

```
mermaid
flowchart LR
  A[KEV Data] -->|Pull| B[PMDB]
  B -->|Process and export| C[Bucket]
  C -->|Pull| D[GitLab Instance]
```

Motivation

The classic approach to vulnerability prioritization is using severity based on CVSS.

This approach provides some guidance, but is too unrefined - more than half of all published CVEs have a high or critical score.

Other metrics need to be employed to reduce remediation fatigue and help developers prioritize their work better. KEV provides a focused list of vulnerabilities that are actively exploited in the wild.

These vulnerabilities also likely have documented exploitation techniques which attackers may reproduce. Many vulnerabilities marked as critical will never be exploited; KEV defines a list of ones that have been exploited and likely will be exploited again.

Combined with existing prioritization methods, KEV helps to focus remediation efforts on the most immediate threats and reduce overall remediation workload.

By adding KEV data to the information presented to users, we deliver these benefits to the GitLab platform, enabling more effective and efficient vulnerability management.

Additionally, FCEB (federal civilian executive branch) agencies must address vulnerabilities in the KEV catalog under BOD 22-01.

While this directive doesn't apply to other government levels or industry, they

often follow federal cybersecurity guidance, making it relevant to a wider audience.

Goals

- Enable users to use KEV data on GitLab as another metric for their vulnerability prioritization efforts.

Phase 1 (MVC)

- Enable access to KEV status through GraphQL API.

Phase 2

- Show KEV information in vulnerability report and details pages.

Phase 3

- Allow filtering vulnerabilities based on KEV status.
- Allow creating policies based on KEV status.

Non-Goals

- Dictate priority to users based on KEV (or any other metric).

Proposal

Support KEV on the GitLab platform.

Following the discussions in the KEV epic, the proposed flow is:

1. PMDB database is extended with a new table to store KEV data.
1. PMDB infrastructure runs the KEV feeder daily to pull, process, and publish KEV data.
1. The advisory-processor receives the KEV data and stores them to the PMDB DB.
1. The exporter combines KEV and EPSS data into a single CVE enrichments dataset, exporting it to a dedicated bucket.
1. GitLab instances pull the CVE enrichments data from the bucket.
1. GitLab instances store KEV and EPSS data in a new pmcveenrichments table.
1. GitLab instances expose KEV status through GraphQL API and present data in vulnerability report and details pages.

mermaid

flowchart LR

```
AF[Feeder] -->|pulls| A[KEV Source]
AF -->|publishes| AP[Advisory Processor]
AP -->|stores| DD[PMDB database]
E[Exporter] -->|loads| DD
```

E --> |exports| B[CVE Enrichment Public Bucket]
GitLab[GitLab instance] --> |syncs| B
GitLab --> |stores| GitLabDB[(pmcveenrichments)]

Design and implementation details

Decisions

- 001: Unify KEV and EPSS as CVE Enrichments

Important note

- Though the KEV catalog is updated with new entries, the status of exploited vulnerabilities rarely changes.
However, vulnerabilities can be removed from the catalog, and CISA doesn't guarantee immutability here. Therefore, we should support this edge case as well.

PMDB

- Create a new KEV table in PMDB with an advisory identifier. This includes changing the schema and any necessary migrations.
- Ingest KEV data into this new PMDB table. Make sure to remove advisories that do not exist in the JSON and have been removed from the catalog.
- Export KEV data along with EPSS data as a unified CVE enrichments dataset.
- Add new pubsub topics to deployment to be used by PMDB components, using existing terraform modules.

GitLab Rails backend

- Create a new pmcveenrichments table in the GitLab database to store unified KEV and EPSS data.
- Configure Rails sync to ingest the unified CVE enrichments data and save to the new table.
- Include KEV status attribute in GraphQL API Occurrence objects, querying from the pmcveenrichments table.

GitLab UI

- Add KEV status to vulnerability report page.
- Add KEV status to vulnerability details page.
- Allow filtering by KEV status.
- Allow creating policies based on KEV status.

Glossary

- PMDB (Package metadata database, also known as License DB): PMDB is a standalone service (and not solely a database), outside of the Rails application, that gathers, stores and exports packages metadata for GitLab instances to consume.
See complete documentation.
PMDB components include:
 - Feeder: a scheduled job called by the PMDB deployment to publish data from the relevant sources to pub/sub messages consumed by PMDB processors.
 - Advisory processor: Runs as a Cloud Run instance and consumes messages published by the advisory feeder containing advisory related data and stores them to the PMDB database.
 - PMDB database: a PostgreSQL instance storing license and advisory data.
 - Exporter: exports license/advisory data from the PMDB database to public GCP buckets.
- GitLab database: the database used by GitLab instances.
- CVE (Common Vulnerabilities and Exposures): a list of publicly known information-security vulnerabilities. "A CVE" usually refers to a specific vulnerability and its CVE ID.
- CISA (Cybersecurity and Infrastructure Security Agency) is a U.S. agency focused on cybersecurity and infrastructure protection.
- KEV (Known Exploited Vulnerabilities): a catalog of actively exploited vulnerabilities maintained by CISA

SECTION: engineering/architecture/design-
documents/kev/decisions/001_unify_kev_and_epss_as_cve_enrichments.md

Context

We are implementing support for KEV data in GitLab.
Initially, we considered storing KEV data separately from EPSS data.
However, after discussion,
we realized that unifying KEV and EPSS data as CVE
enrichments could provide benefits in terms of performance and simplicity.

The unified approach simplifies data management and querying. By consolidating KEV and EPSS data into a single table, we create a more efficient structure for these CVE-based enrichments. This design allows for more efficient DB queries by reducing the number of joins.

It's important to note that while KEV data includes around 1,000 records, EPSS data includes around 250,000 records (these numbers will increase over time). KEV data is limited to a boolean value indicating whether a vulnerability is known to be exploited.
The small size of this additional boolean field and the relatively small number of KEV records compared to EPSS records support the decision to combine these datasets, as it adds minimal overhead to the existing EPSS data structure.

The current EPSS implementation already has a design in place, which we can leverage and extend for this unified approach. In addition, the API remains unaffected by these backend changes, maintaining its current structure and usability.

Decision

We have decided to:

1. Maintain separate tables for KEV and EPSS data in PMDB.
2. Export both KEV and EPSS data into a single, shared GCP bucket.
3. Implement a single process for syncing and ingesting the unified data into a new `pmcveenrichments` table in GitLab DB.
4. Structure the `pmcveenrichments` table to include:
 - `cveid`
 - `epssscore` (nullable)
 - `hasknownexploit` (nullable)
5. Use GraphQL queries to access enrichment data exclusively from the `pmcveenrichments` table.

Consequences

Positive

- Enhanced SQL query performance for CVE enrichment data retrieval by storing EPSS and KEV in a unified table, requiring only a single join instead of two.
- Reduces the complexity of the data model on the backend.
- Eliminates code duplication by using shared feature flags, sync, ingestion, and export processes for both KEV and EPSS data.
- Minimal impact on upsert queries and the Write Ahead Log since KEV data is restricted to a single boolean column.

Negative

- Loss of cursor functionality for KEV, requiring full data export each time due to EPSS data always being fully exported.
- One-time performance cost during PMDB's export due to joining KEV and EPSS tables.

Alternatives

1. Store KEV data separately:
 - Advantages:
 - Maintains separation of concerns.
 - Preserves cursor functionality for KEV.
 - Less frequent KEV data updates in GitLab DB (due to smaller dataset and less frequent changes compared to EPSS).
 - Reduces the Write Ahead Log during the ingestion of KEV data and EPSS data.
 - Disadvantages:

- GraphQL queries trigger SQL queries that JOIN two tables.
- Increased complexity in ingestion and synchronizing two separate datasets.

2. Denormalize KEV data into vulnerabilities tables:

- Advantages:
 - Fast access to KEV data when querying vulnerabilities.
- Disadvantages:
 - Data redundancy.
 - Different approach between EPSS and KEV, leading to inconsistent data management.
 - Duplicated ingestion and sync processes.

3. Store KEV data in pmadvisories:

- Advantages:
 - Keeps advisory-related data together.
 - Leverages current exporting, ingestion, and sync processes for advisories, lessening the need for new implementations.
- Disadvantages:
 - Requires joining with advisories for GraphQL querying, which may impact performance.
 - May introduce complexity if KEV data needs to be accessed independently of advisory data.

SECTION: engineering/architecture/design-documents/knowledge_graph/_index.md

{{< engineering/design-document-header >}}

Summary

This document proposes a new service that enhances repository insights by creating and maintaining graph databases for repositories. Using Kuzu, a file-embedded graph database, the service will store each repository's knowledge graph separately, enabling users to query relationships between directories, files, classes, functions, and dependencies through Cypher queries. The architecture includes a scalable approach with primary nodes and replicas for high availability, with all requests routed through GitLab Rails for authentication and authorization. This feature will enable Duo Chat to answer complex repository questions (like "where is method X defined" or "what dependencies are used in file Y"), while also supporting non-AI use cases such as repository statistics and related file discovery when browsing code.

Motivation

Currently, we don't have a tool which would provide enhanced repository knowledge. We are able to search a repository code, but that's all. We don't have a tool which could be used to get better insight into repository - what

entities (directories, files, classes, functions) are defined in the repository and how are these entities connected with each other. Knowledge graph provides this information.

For AI it will be especially useful for Duo Chat to answer user's questions about repository (for example "where is defined XYZ method", "from where we call method XYZ", "what dependencies are used in a file", "are there any unused methods", "what are commonly used functions"). So one of primary use-cases will be usage in Duo Chat. LLMs are able to build Cypher query tailored specifically for answering user's question if LLM knows the schema of graph database.

Having a knowledge graph will also be useful for non-AI use-cases - e.g. for repository statistics or showing related files when browsing through repository.

Goals

- Make available insight information about repository to other services and relationships between entities in the repository. For now we focus on creating knowledge graph primarily for source code repositories:
 - entities defined in the repository such as directories, files, classes, functions, dependencies
 - relationships between these entities - where is each class or function defined, what methods are defined in a class, what functions are called from another function, what dependencies are used in a file...
- But in future iterations we can expand knowledge graph beyond the source code (and track also other relationships - relationships with issues, merge requests and other entities).
- Allow querying graph DB with LLM-generated Cypher queries

Non-Goals

- Implementation of repository parser. For more details on the parser, see the Knowledge Graph First iteration.
For purposes of this document, the expectation is that the repository parser will be either a library or a standalone application which will be called on graph nodes. It will accept repository files on input and produce parsed data (graph nodes and edges) in a format accepted by graph DB service, for example CSV or JSON files.

Proposal

- Store knowledge graphs for repositories in file-embedded Kuzu DBs (each repository will have its own graph DB)
- Build a thin API service which runs on graph nodes and which takes care of serving incoming query requests (and also takes care of DB management tasks).
- To avoid building graph nodes infrastructure from scratch, we will use Zoekt nodes for indexing and querying also graph databases. We will extend the existing gitlab-zoekt service instead of creating a separate new service.
- Create an abstraction layer on GitLab Rails side which can be used by other

services to query graph databases using Cypher query

Design and implementation details

Because there is already Exact Code Search (Zoekt) which uses similar architecture as knowledge graph, it would be best to make the existing Zoekt infrastructure more generic so it will support both Zoekt searching and graph database searching. Then we can deploy graph database together with Zoekt on the same nodes. The major benefit is that we can re-use existing Zoekt logic (nodes management on Rails side) and infrastructure (deployment of Zoekt nodes) and node logic itself (Zoekt Webservice and Indexer).

mermaid

flowchart TD

subgraph GitLab Rails

T1[Duo Chat Tool] -->|Cypher query for project X| K(Knowledge graph layer)

T2[Other service] -->|Cypher query for project Y| K(Knowledge graph layer)

end

K --> |Cypher query for project X| GA1(zoekt-webservice)

K --> |Cypher query for project Y| GA2(zoekt-webservice)

subgraph zoekt node 1

GA1 -->K1[Kuzu DB X]

GA1 -->K2[Kuzu DB A]

GA1 -->K3[Kuzu DB B]

end

subgraph zoekt node 2

GA2 -->K21[Kuzu DB Y]

GA2 -->K22[Kuzu DB C]

GA2 -->K23[Kuzu DB D]

end

Graph node (Zoekt node)

Each repository will be stored in a separate Kuzu graph database. Kuzu is a file-embedded graph DB, each graph is stored in a directory. Because Kuzu is an embeddable file DB, we will implement a simple API layer which accepts requests from GitLab Rails server, opens repository graph DB and executes a query. Because we plan to re-use existing gitlab-zoekt service, this graph DB API layer will be added to gitlab-zoekt API.

Kuzu database will be used also on client side, so in future we can consider also